WEEK 8

ViewModel, UI State & MVVM

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

This is one of the most important weeks for your job. You'll learn how professional Android teams **structure a screen**: a **ViewModel** holds the screen's state and logic, exposes it as a **StateFlow** of **UiState**, and the composable just shows it. This is called **MVVM** with **unidirectional data flow**. At the end you'll rebuild the Quiz app the way a team would.

Day	Topic	You will be able to...
Day 1	Why ViewModel? MVVM	Explain the architecture and write your first ViewModel
Day 2	StateFlow	Expose and observe state with StateFlow
Day 3	Designing UiState	Model everything a screen shows in one state object
Day 4	Route + Screen and the ViewModel lifecycle	Connect ViewModels to screens and navigation correctly
Day 5	Loading, errors and one-off events	Handle async work, messages and navigation after actions
Day 6	Weekly project	Rebuild the Quiz app with a ViewModel

> **Tip:** Compose and ViewModel code is **compile-checked** (ViewModel and lifecycle APIs against stand-ins that copy the real API). The StateFlow examples on Day 2 were **run** with the real kotlinx.coroutines library, and the output is real.

WEEK 8 • DAY 1

Why ViewModel? MVVM

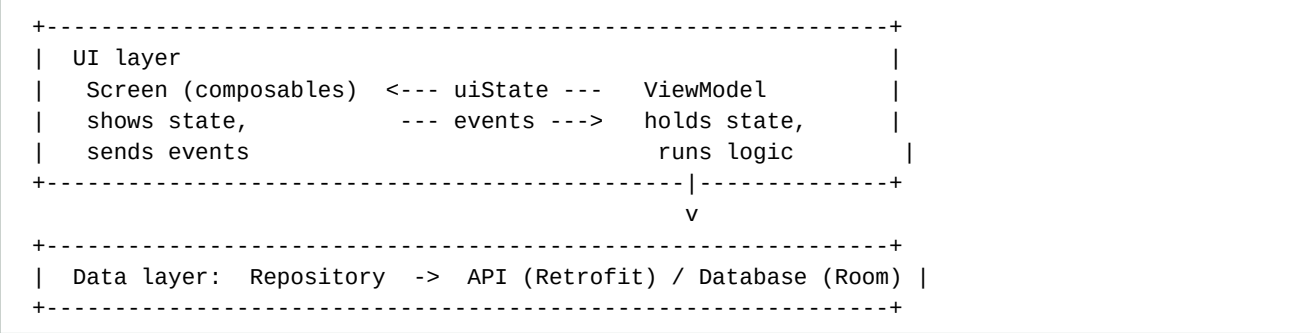
Today's goal: Understand the problems ViewModels solve, the MVVM architecture, and write your first ViewModel.

1. The problems with state in composables

Problem	Example
State is lost on rotation (with remember)	The quiz restarts when the phone turns
rememberSaveable can only store small, simple data	A list of 500 products shouldn't go in a Bundle
Logic mixed with UI is hard to test	You can't test the scoring without a phone
Work restarts on recomposition	A network call inside a composable could run many times
Screens get huge	One file with UI, validation, networking and formatting

2. MVVM: separate the jobs

Sketch: The layers of an Android app

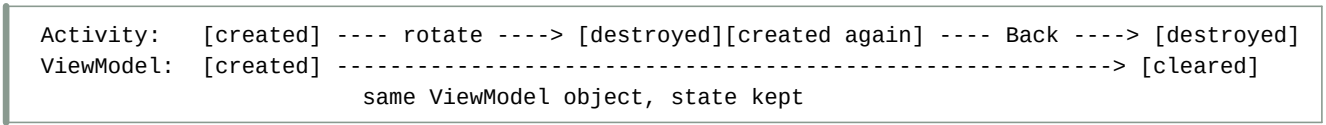


Part	Job	Knows about
Model (data layer)	Getting and saving data	API, database. Nothing about the UI
ViewModel	Holding screen state, handling user actions	The data layer. Not about composables or Context
View (composables)	Drawing the state, reporting events	The UiState and event functions

3. What a ViewModel gives you

- It **survives configuration changes** such as rotation. The same ViewModel object is reused when the Activity is recreated.
- It has a **viewModelScope** for background work, cancelled automatically when the screen is closed for good.
- It's a normal Kotlin class, so its logic can be **unit tested** on your computer, without a phone (Week 14).

Sketch: A ViewModel outlives rotation



4. Your first ViewModel

CounterViewModel with Compose state

```
class CounterViewModel : ViewModel() {
    var count by mutableStateOf(0)
        private set                     // the UI can read it, only the ViewModel changes it

    fun increment() {
        count++
    }

    fun reset() {
        count = 0
    }
}

@Composable
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
    Column(modifier = Modifier.padding(24.dp), horizontalAlignment = Alignment.CenterHorizontally) {
        Text("Count: ${viewModel.count}", style = MaterialTheme.typography.headlineMedium)
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Button(onClick = { viewModel.increment() }) { Text("Add") }
            OutlinedButton(onClick = { viewModel.reset() }) { Text("Reset") }
        }
    }
}
```

Code	Meaning
class CounterViewModel : ViewModel()	Inherits from Android's ViewModel class (Week 3 inheritance)
private set	Encapsulation (Week 3, Day 2): the screen can read count but can't change it directly
fun increment()	An **event**: the screen asks the ViewModel to do something
viewModel()	Gets the ViewModel for this screen, creating it the first time. After rotation, you get the **same** object back

Dependency needed for viewModel() (Android only)

```
// libs.versions.toml
// androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "lifecycleRuntimeKtx" }
dependencies {
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    implementation(libs.androidx.lifecycle.runtime.compose)    // for collectAsStateWithLifecycle (Day 2)
}
```

Rotate the phone: the count stays! The ViewModel kept it, even though remember wasn't used.

5. Rules for ViewModels

Do	Don't
Hold the screen's state and logic	Hold references to composables, Views or an Activity (memory leaks!)
Expose read-only state	Let the UI change the state directly
Offer one function per user action: onLoginClick(), onEmailChange()	Put Android Context into it (use AndroidViewModel only if really needed)
Get data from repositories	Call the API directly from the ViewModel in big apps

Practice Exercises

1. Create a **LikesViewModel** with likes (private set) and like() / unlike() functions, and a screen that uses it. Rotate to check it survives.

2. Move the Week 6 WaterTracker state into a **WaterViewModel**, with addGlass() and reset().

3. Explain in your own words why a ViewModel must not keep a reference to an Activity.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does a ViewModel survive that remember doesn't?

Q2. What are the three parts of MVVM?

Q3. Why use private set on ViewModel state?

Q4. What does viewModel() return after rotation?

Q5. Why shouldn't a ViewModel hold an Activity?

WEEK 8 • DAY 2

StateFlow

Today's goal: Expose state with StateFlow, update it safely, and observe it from Compose.

1. What is StateFlow?

A **StateFlow** is a container that always holds **one current value** and lets others **observe** it: every time the value changes, observers get the new value. It comes from **Kotlin coroutines**, so it isn't tied to Compose, and it's the standard way ViewModels expose state in professional apps.

StateFlow basics (plain Kotlin, run with kotlinx.coroutines)

```
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update

class Counter {
    private val _count = MutableStateFlow(0)          // private and changeable
    val count: StateFlow<Int> = _count.asStateFlow()   // public and read-only

    fun increment() {
        _count.update { it + 1 }                       // safe update based on the current value
    }
}

fun main() {
    val counter = Counter()
    println("Start: ${counter.count.value}")
    counter.increment()
    counter.increment()
    println("After two increments: ${counter.count.value}")
}
```

Output

```
Start: 0
After two increments: 2
```

Code	Meaning
MutableStateFlow(0)	A StateFlow you can change, starting at 0
private val _count	The underscore name is a convention: the private, mutable version
val count: StateFlow<Int> = _count.asStateFlow()	The public, **read-only** version for the UI
_count.value	Read or set the current value directly
_count.update { it + 1 }	Set a new value calculated from the current one. Safe even if two things update at once

Tip: This **_private / public** pair is called a **backing property**. You'll see it in nearly every ViewModel in your team's code.

2. Observing a StateFlow

Collecting values (plain Kotlin, run with kotlinx.coroutines)

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    val temperature = MutableStateFlow(25)

    val job = launch {
        temperature.collect { value ->         // runs for the current value and every change
            println("Temperature is now $value")
        }
    }

    delay(100)
    temperature.value = 28
    delay(100)
    temperature.value = 28                     // same value: StateFlow ignores it
    delay(100)
    temperature.value = 31
    delay(100)
    job.cancel()                               // stop observing
    println("Done")
}
```

Output

```
Temperature is now 25
Temperature is now 28
Temperature is now 31
Done
```

Notice that setting the **same value** (28 again) didn't notify anyone. StateFlow only emits real **changes**. In Compose, you don't call collect yourself. You use **collectAsStateWithLifecycle()**.

3. StateFlow in a ViewModel and Compose

CounterViewModel with StateFlow

```
class CounterViewModel : ViewModel() {
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count.asStateFlow()

    fun increment() = _count.update { it + 1 }
    fun reset() { _count.value = 0 }
}

@Composable
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
    val count by viewModel.count.collectAsStateWithLifecycle()

    Column(modifier = Modifier.padding(24.dp)) {
        Text("Count: $count", style = MaterialTheme.typography.headlineMedium)
        Button(onClick = viewModel::increment) { Text("Add") }
        TextButton(onClick = viewModel::reset) { Text("Reset") }
    }
}
```

Code	Meaning
collectAsStateWithLifecycle()	Turns the StateFlow into Compose State. The screen recomposes when it changes, and collection pauses while the app is in the background (saves battery)
val count by ...	Delegation again, so you use count directly
viewModel::increment	A function reference (Week 2) passed as the onClick lambda

4. Compose state vs StateFlow in ViewModels

mutableStateOf in the ViewModel	StateFlow in the ViewModel
Simpler to write	A few more lines
Ties the ViewModel to Compose	Plain Kotlin: works with Compose, Views and tests
Fine for small apps and learning	The standard in most teams, and in Google's samples

From now on, this book uses **StateFlow**. If your team uses mutableStateOf in ViewModels, that works too. Follow the team.

Practice Exercises

1. Write a plain-Kotlin **Thermostat** class with a private MutableStateFlow for the temperature and up()/down() functions that keep it between 16 and 30. Test it in main().

2. Convert your LikesViewModel from Day 1 to use StateFlow and collectAsStateWithLifecycle.

3. In the collect example, set the value to 25, 25, 26. How many lines are printed? Why?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between MutableStateFlow and StateFlow?

Q2. Why expose asStateFlow() publicly instead of the MutableStateFlow?

Q3. What does update { } do?

Q4. Does StateFlow emit when you set the same value again?

Q5. How do you observe a StateFlow in Compose?

WEEK 8 • DAY 3

Designing UiState

Today's goal: Model everything a screen shows in a single immutable UiState, and update it with copy().

1. One state object per screen

Instead of many separate StateFlows (email, password, isLoading, error...), teams put everything a screen shows into one **data class**, the **UiState**. The ViewModel exposes **one StateFlow<UiState>**.

Many separate flows	One UiState
email, password, isLoading, errorMessage: 4 flows	data class LoginUiState(email, password, isLoading, errorMessage)
Easy to forget updating one	One update changes everything consistently
Screen collects 4 things	Screen collects 1 thing

2. A login screen's UiState and ViewModel

LoginUiState and LoginViewModel

```
data class LoginUiState(
    val email: String = "",
    val password: String = "",
    val isLoading: Boolean = false,
    val errorMessage: String? = null
) {
    // derived values belong in the state class (or are calculated by the ViewModel)
    val canSubmit: Boolean
        get() = email.contains("@") && password.length >= 6 && !isLoading
}

class LoginViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    fun onEmailChange(email: String) {
        _uiState.update { it.copy(email = email.trim(), errorMessage = null) }
    }

    fun onPasswordChange(password: String) {
        _uiState.update { it.copy(password = password, errorMessage = null) }
    }

    fun onLoginClick() {
        val state = _uiState.value
        if (state.password == "wrongpass") {
            _uiState.update { it.copy(errorMessage = "Incorrect email or password") }
        } else {
            _uiState.update { it.copy(isLoading = true) }
            // Day 5: call the repository here, then update isLoading/errorMessage
        }
    }
}
```

Pattern	Why
All properties are **val** with default values	The state is immutable; the defaults are the initial screen
it.copy(email = ...)	Create a new state with one change; everything else stays the same (Week 3 data classes)
canSubmit is a computed property	Derived from other fields, so it can never be out of sync
One function per event: onEmailChange, onLoginClick	Clear names, easy to test, easy to read in the screen
errorMessage = null when the user types	The error disappears as soon as the user starts fixing it

3. The screen just displays the state

LoginScreen (stateless)

```
data class LoginUiState(
    val email: String = "",
    val password: String = "",
    val isLoading: Boolean = false,
    val errorMessage: String? = null
) {
    val canSubmit: Boolean get() = email.contains("@") && password.length >= 6 && !isLoading
}

@Composable
fun LoginScreen(
    uiState: LoginUiState,
    onEmailChange: (String) -> Unit,
    onPasswordChange: (String) -> Unit,
    onLoginClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier.fillMaxWidth().padding(24.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text("Log in", style = MaterialTheme.typography.headlineMedium)
        OutlinedTextField(
            value = uiState.email, onValueChange = onEmailChange,
            label = { Text("Email") }, singleLine = true, modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = uiState.password, onValueChange = onPasswordChange,
            label = { Text("Password") }, singleLine = true,
            visualTransformation = PasswordVisualTransformation(), modifier = Modifier.fillMaxWidth()
        )
        if (uiState.errorMessage != null) {
            Text(uiState.errorMessage, color = MaterialTheme.colorScheme.error)
        }
        Button(onClick = onLoginClick, enabled = uiState.canSubmit, modifier = Modifier.fillMaxWidth()) {
            if (uiState.isLoading) {
                CircularProgressIndicator(Modifier.size(20.dp), strokeWidth = 2.dp)
            } else {
                Text("Log in")
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun LoginScreenErrorPreview() {
    LoginScreen(
        uiState = LoginUiState(email = "anjana@example.com", password = "wrongpass",
            errorMessage = "Incorrect email or password"),
        onEmailChange = { }, onPasswordChange = { }, onLoginClick = { }
    )
}
```

Because the screen only receives **uiState** and **event lambdas**, you can preview **any** situation (error, loading, filled in) just by creating a LoginUiState. No ViewModel, no network.

4. Sealed UiState for very different states

When the whole screen changes

```
data class Article(val id: Int, val title: String, val body: String)

sealed interface ArticleUiState {
    data object Loading : ArticleUiState
    data class Success(val article: Article) : ArticleUiState
    data class Error(val message: String) : ArticleUiState
}

@Composable
fun ArticleScreen(uiState: ArticleUiState, onRetry: () -> Unit) {
    when (uiState) {
        ArticleUiState.Loading -> Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
        is ArticleUiState.Error -> Column(Modifier.padding(24.dp)) {
            Text(uiState.message)
            Button(onClick = onRetry) { Text("Retry") }
        }
        is ArticleUiState.Success -> Column(Modifier.padding(24.dp)) {
            Text(uiState.article.title, style = MaterialTheme.typography.headlineSmall)
            Text(uiState.article.body)
        }
    }
}
```

Use a data class UiState when...	Use a sealed UiState when...
The screen always shows the same parts (a form)	The screen shows completely different things per state
Several things can be true at once (loading AND has old data)	Exactly one state at a time

Practice Exercises

1. Design a **ProfileUiState** for a profile edit screen: name, bio, isSaving, errorMessage and a computed canSave (name not blank).

2. Write a **ProfileViewModel** with onNameChange, onBioChange (max 150 characters) and onSaveClick.

3. Write a stateless ProfileScreen and two previews: an empty state and a saving state.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why put a screen's state in one data class?

Q2. Why are UiState properties val?

Q3. Where should a value like canSubmit come from?

Q4. When is a sealed UiState better than a data class?

Q5. Why is the screen easy to preview?

WEEK 8 • DAY 4

Route + Screen and the ViewModel Lifecycle

Today's goal: Connect ViewModels to screens and navigation the way teams do, and know when ViewModels are created and destroyed.

1. The Route + Screen split

Stateful Route, stateless Screen

```
data class NotesUiState(val notes: List<String> = emptyList(), val draft: String = "")

class NotesViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(NotesUiState())
    val uiState: StateFlow<NotesUiState> = _uiState.asStateFlow()

    fun onDraftChange(text: String) = _uiState.update { it.copy(draft = text) }

    fun onAddClick() = _uiState.update {
        if (it.draft.isBlank()) it else it.copy(notes = it.notes + it.draft.trim(), draft = "")
    }

    fun onDelete(note: String) = _uiState.update { it.copy(notes = it.notes - note) }
}

// 1. Route: small, connects the ViewModel. No UI here
@Composable
fun NotesRoute(viewModel: NotesViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    NotesScreen(
        uiState = uiState,
        onDraftChange = viewModel::onDraftChange,
        onAddClick = viewModel::onAddClick,
        onDelete = viewModel::onDelete
    )
}

// 2. Screen: all the UI, stateless and previewable
@Composable
fun NotesScreen(
    uiState: NotesUiState,
    onDraftChange: (String) -> Unit,
    onAddClick: () -> Unit,
    onDelete: (String) -> Unit
) {
    Column(Modifier.fillMaxSize().padding(16.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            OutlinedTextField(uiState.draft, onDraftChange, label = { Text("New note") },
                modifier = Modifier.weight(1f), singleLine = true)
            IconButton(onClick = onAddClick) { Icon(Icons.Default.Add, contentDescription = "Add note") }
        }
        LazyColumn {
            items(uiState.notes, key = { it }) { note ->
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Text(note, Modifier.weight(1f).padding(vertical = 12.dp))
                    IconButton(onClick = { onDelete(note) }) {
                        Icon(Icons.Default.Delete, contentDescription = "Delete $note")
                    }
                }
            }
        }
    }
```

```
}

@Preview(showBackground = true)
@Composable
fun NotesScreenPreview() {
    NotesScreen(NotesUiState(notes = listOf("Buy milk", "Call Amma"), draft = "Study"), { }, { }, { })
}
```

Tip: Why two composables? The Screen can be previewed and tested with any UiState. The Route is tiny, so it hardly needs testing. Most teams name them **XxxRoute** and **XxxScreen**.

2. Plugging the Route into navigation

AppNavHost calls Routes

```
@Serializable object NotesListRoute
@Serializable data class NoteDetailRoute(val noteId: Int)

@Composable
fun AppNavHost() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = NotesListRoute) {
        composable<NotesListRoute> {
            NotesListRoute(onNoteClick = { id -> navController.navigate(NoteDetailRoute(id)) })
        }
        composable<NoteDetailRoute> {
            NoteDetailRoute(onBack = { navController.popBackStack() })   // the ViewModel reads the ID itself
        }
    }
}

@Composable
fun NotesListRoute(onNoteClick: (Int) -> Unit) {
    Button(onClick = { onNoteClick(7) }) { Text("Open note 7") }
}

@Composable
fun NoteDetailRoute(onBack: () -> Unit) {
    TextButton(onClick = onBack) { Text("Back") }
}
```

3. Reading navigation arguments in the ViewModel

A detail screen's ViewModel needs the ID from the route. Android gives every ViewModel created with viewModel() a **SavedStateHandle** containing the route's arguments:

SavedStateHandle + toRoute()

```
@Serializable data class NoteDetailRoute(val noteId: Int)

data class NoteDetailUiState(val title: String = "", val isLoading: Boolean = true)

class NoteDetailViewModel(savedStateHandle: SavedStateHandle) : ViewModel() {
    private val noteId: Int = savedStateHandle.toRoute<NoteDetailRoute>().noteId

    private val _uiState = MutableStateFlow(NoteDetailUiState())
    val uiState: StateFlow<NoteDetailUiState> = _uiState.asStateFlow()

    init {
        // load the note with this ID (from a repository in a real app)
        _uiState.value = NoteDetailUiState(title = "Note number $noteId", isLoading = false)
    }
}
```

The screen never passes the ID to the ViewModel by hand: the ViewModel reads it from the route. This is also what keeps the screen working if Android kills and restores the app.

4. The ViewModel lifecycle

Event	What happens to the ViewModel
The screen is opened (first viewModel() call)	A new ViewModel is created; init { } runs
Rotation, dark mode change, language change	**The same ViewModel is kept**; state survives
You navigate forward to another screen	Kept (the screen is still in the back stack)
The user presses Back and the screen is removed	**Cleared**: onCleared() runs, viewModelScope is cancelled
Android kills the app in the background	Lost (only SavedStateHandle data is restored)

With Navigation, each destination (back stack entry) gets **its own** ViewModel. Two copies of the same screen in the back stack have two different ViewModels.

> **Careful:** A ViewModel is **not permanent storage**. When the user closes the screen or the app is killed, the data is gone. To keep data permanently, save it to a database (Room, Week 11) or DataStore.

5. ViewModels with dependencies

Real ViewModels need a repository. With plain viewModel() you'd need a **factory**. Most teams use **Hilt** (Week 12), which makes it as easy as `hiltViewModel()`. Until then, give the constructor a default value:

A default dependency (simple approach until Hilt)

```
class TaskRepository {
    fun getTasks(): List<String> = listOf("Learn ViewModel", "Build the quiz")
}

class TasksViewModel(
    private val repository: TaskRepository = TaskRepository()
) : ViewModel() {
    private val _tasks = MutableStateFlow(repository.getTasks())
    val tasks: StateFlow<List<String>> = _tasks.asStateFlow()
}
```

Practice Exercises

1. Split one of your Week 6 screens into an **XxxRoute** with a ViewModel and a stateless **XxxScreen** with a preview.
2. Create **ProductDetailViewModel(savedStateHandle)** that reads ProductDetailRoute(productId) and shows "Product N".
3. Add Log.d calls in a ViewModel's init and onCleared(). Rotate the phone, then press Back. When does each one appear in Logcat?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the job of an XxxRoute composable?

Q2. How does a detail ViewModel get the ID from the route?

Q3. Is the ViewModel cleared when you rotate the phone?

Q4. Is a ViewModel a good place for permanent data?

Q5. What runs when a ViewModel is cleared?

WEEK 8 • DAY 5

Loading, Errors & One-Off Events

Today's goal: Run background work in viewModelScope, show loading and errors, and handle messages and navigation after actions.

1. Background work with viewModelScope

Loading data takes time. A ViewModel starts this work in **viewModelScope.launch { }**. It runs without freezing the UI, and it's cancelled automatically when the ViewModel is cleared. (Coroutines are explained properly in Week 9.)

Loading data with loading and error states

```
data class Weather(val city: String, val temperature: Int)

class WeatherRepository {
    suspend fun getWeather(city: String): Weather {
        delay(1500)                                        // pretend this is a network call
        if (city.isBlank()) throw IllegalArgumentException("City is empty")
        return Weather(city, 31)
    }
}

data class WeatherUiState(
    val city: String = "Kochi",
    val weather: Weather? = null,
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)

class WeatherViewModel(
    private val repository: WeatherRepository = WeatherRepository()
) : ViewModel() {
    private val _uiState = MutableStateFlow(WeatherUiState())
    val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()

    init { refresh() }

    fun onCityChange(city: String) = _uiState.update { it.copy(city = city) }

    fun refresh() {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {
                val weather = repository.getWeather(_uiState.value.city)
                _uiState.update { it.copy(weather = weather, isLoading = false) }
            } catch (e: Exception) {
                _uiState.update { it.copy(isLoading = false, errorMessage = "Couldn't load the weather") }
            }
        }
    }
}
```

Step	Code
1. Show loading, clear old errors	isLoading = true, errorMessage = null
2. Do the slow work	repository.getWeather(...) (a **suspend** function)
3a. Success: show data, stop loading	weather = weather, isLoading = false
3b. Failure: show an error, stop loading	isLoading = false, errorMessage = "..."

> **Tip:** Always set **isLoading = false** in **both** the success and the error path. A spinner that never stops is a classic bug.

2. Showing loading and errors in the screen

WeatherScreen

```
data class Weather(val city: String, val temperature: Int)
data class WeatherUiState(
    val city: String = "Kochi",
    val weather: Weather? = null,
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)

@Composable
fun WeatherScreen(uiState: WeatherUiState, onCityChange: (String) -> Unit, onRefresh: () -> Unit) {
    Column(Modifier.fillMaxSize().padding(24.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        OutlinedTextField(uiState.city, onCityChange, label = { Text("City") }, singleLine = true)
        Button(onClick = onRefresh, enabled = !uiState.isLoading) { Text("Get weather") }

        when {
            uiState.isLoading -> CircularProgressIndicator()
            uiState.errorMessage != null -> Text(uiState.errorMessage, color = MaterialTheme.colorScheme.error)
            uiState.weather != null -> Text(
                "${uiState.weather.city}: ${uiState.weather.temperature} C",
                style = MaterialTheme.typography.headlineMedium
            )
        }
    }
}
```

3. One-off events: messages

Some things should happen **once**: a snackbar saying "Saved", or navigating after login. Google's recommended way is to put the event **in the UiState**, and have the screen tell the ViewModel when it has been handled:

A message shown once in a Snackbar

```
data class SaveUiState(val text: String = "", val userMessage: String? = null)

class SaveViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(SaveUiState())
    val uiState: StateFlow<SaveUiState> = _uiState.asStateFlow()

    fun onTextChange(text: String) = _uiState.update { it.copy(text = text) }
    fun onSaveClick() = _uiState.update { it.copy(userMessage = "Saved!") }
    fun onMessageShown() = _uiState.update { it.copy(userMessage = null) }
}

@Composable
fun SaveScreen(uiState: SaveUiState, onTextChange: (String) -> Unit,
               onSaveClick: () -> Unit, onMessageShown: () -> Unit) {
    val snackbarHostState = remember { SnackbarHostState() }

    uiState.userMessage?.let { message ->
        LaunchedEffect(message) {
            snackbarHostState.showSnackbar(message)
            onMessageShown()                       // tell the ViewModel: done
        }
    }

    Scaffold(snackbarHost = { SnackbarHost(snackbarHostState) }) { innerPadding ->
        Column(Modifier.padding(innerPadding).padding(16.dp)) {
            OutlinedTextField(uiState.text, onTextChange, label = { Text("Note") })
            Button(onClick = onSaveClick) { Text("Save") }
        }
    }
}
```

Code	Meaning
userMessage: String? = null	null = no message to show
LaunchedEffect(message) { }	Runs a coroutine when message changes, so the snackbar shows exactly once
onMessageShown()	Clears the message, so it won't show again after rotation

4. One-off events: navigation after an action

Navigate after a successful login

```
data class LoginUiState(val isLoading: Boolean = false, val isLoggedIn: Boolean = false)

@Composable
fun LoginScreen(uiState: LoginUiState, onLoginClick: () -> Unit, onLoggedIn: () -> Unit) {
    LaunchedEffect(uiState.isLoggedIn) {
        if (uiState.isLoggedIn) onLoggedIn()       // the NavHost navigates (with popUpTo)
    }
    Button(onClick = onLoginClick, enabled = !uiState.isLoading, modifier = Modifier.padding(24.dp)) {
        Text("Log in")
    }
}
```

The ViewModel sets **isLoggedIn = true** after a successful login. The screen reacts and calls onLoggedIn, and the NavHost does the navigation. The ViewModel still knows nothing about navigation.

> **Tip:** You may also see one-off events sent through a **Channel** or **SharedFlow** in some codebases. It works too. Follow your team's pattern.

Practice Exercises

1. Make the LoginViewModel from Day 3 call a fake repository (delay(1000)) that fails for password "wrongpass". Show loading and errors.

2. Add a "Retry" button to the WeatherScreen error state.

3. After a successful login, show a snackbar "Welcome back!" using the userMessage pattern.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why use viewModelScope.launch for slow work?

Q2. What must you remember in both the success and the error path?

Q3. How do you show a snackbar only once with the UiState pattern?

Q4. Who performs navigation after login: the ViewModel or the screen?

Q5. What does LaunchedEffect(key) do?

WEEK 8 • DAY 6

Weekly Project: Quiz App with a ViewModel

Today's goal: Rebuild the Week 7 Quiz app with a QuizUiState and QuizViewModel, the way a team would.

1. What changes compared to Week 7?

Week 7 (state in composables)	Week 8 (ViewModel)
index, answers, selected: rememberSaveable in QuizScreen	Everything in one QuizUiState inside QuizViewModel
Scoring done when navigating	The ViewModel updates the score after every answer
QuizScreen is stateful	QuizScreen is stateless: previews for any state
Logic mixed with UI	Logic in the ViewModel, testable without a phone

2. UiState and ViewModel

QuizViewModel.kt

```
data class Question(val text: String, val options: List<String>, val correctIndex: Int)

val quizQuestions = listOf(
    Question("Which class survives rotation?", listOf("Activity", "ViewModel", "Composable", "Intent"), 1),
    Question("What exposes read-only state?", listOf("MutableStateFlow", "StateFlow", "var", "remember"), 1),
    Question("Where should business logic live?", listOf("Composable", "MainActivity", "ViewModel", "Theme.kt"), 2),
    Question("How do you update a data class state?", listOf("copy()", "set()", "new", "clone()"), 0)
)

data class QuizUiState(
    val questions: List<Question> = quizQuestions,
    val currentIndex: Int = 0,
    val selectedOption: Int? = null,
    val score: Int = 0,
    val isFinished: Boolean = false
) {
    val currentQuestion: Question get() = questions[currentIndex]
    val progress: Float get() = (currentIndex + 1) / questions.size.toFloat()
    val isLastQuestion: Boolean get() = currentIndex == questions.lastIndex
}

class QuizViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(QuizUiState())
    val uiState: StateFlow<QuizUiState> = _uiState.asStateFlow()

    fun onOptionSelected(index: Int) {
        _uiState.update { it.copy(selectedOption = index) }
    }

    fun onNextClick() {
        _uiState.update { state ->
            val selected = state.selectedOption ?: return@update state
            val newScore = if (selected == state.currentQuestion.correctIndex) state.score + 1 else state.score
            if (state.isLastQuestion) {
                state.copy(score = newScore, isFinished = true)
            } else {
                state.copy(score = newScore, currentIndex = state.currentIndex + 1, selectedOption = null)
            }
        }
    }

    fun onRestart() {
        _uiState.value = QuizUiState(questions = quizQuestions.shuffled())
```

```
    }
}
```

Code	Why
currentQuestion, progress, isLastQuestion as computed properties	Derived from the state, never out of sync
val selected = state.selectedOption ?: return@update state	If nothing is selected, keep the state unchanged (Elvis with a labelled return)
state.copy(...) in every branch	A new immutable state each time
onRestart() shuffles the questions	A new game is just a new QuizUiState

3. Route and Screen

QuizScreen.kt

```
@Composable
fun QuizRoute(viewModel: QuizViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    QuizScreen(
        uiState = uiState,
        onOptionSelected = viewModel::onOptionSelected,
        onNextClick = viewModel::onNextClick,
        onRestart = viewModel::onRestart
    )
}

@Composable
fun QuizScreen(
    uiState: QuizUiState,
    onOptionSelected: (Int) -> Unit,
    onNextClick: () -> Unit,
    onRestart: () -> Unit
) {
    if (uiState.isFinished) {
        QuizResult(score = uiState.score, total = uiState.questions.size, onRestart = onRestart)
        return
    }
    val question = uiState.currentQuestion
    Column(Modifier.fillMaxSize().padding(24.dp)) {
        Text("Question ${uiState.currentIndex + 1} of ${uiState.questions.size}",
            style = MaterialTheme.typography.labelLarge)
        LinearProgressIndicator(progress = { uiState.progress },
            modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp))
        Text(question.text, style = MaterialTheme.typography.headlineSmall)
        Spacer(Modifier.height(16.dp))
        question.options.forEachIndexed { index, option ->
            Row(
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.fillMaxWidth().clickable { onOptionSelected(index) }.padding(vertical = 4.dp)
            ) {
                RadioButton(selected = uiState.selectedOption == index, onClick = { onOptionSelected(index) })
                Text(option)
            }
        }
        Spacer(Modifier.weight(1f))
        Button(onClick = onNextClick, enabled = uiState.selectedOption != null, modifier = Modifier.fillMaxWidth()) {
            Text(if (uiState.isLastQuestion) "Finish" else "Next")
        }
    }
}

@Composable
fun QuizResult(score: Int, total: Int, onRestart: () -> Unit) {
    Column(
        Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
```

```
        Text("$score / $total", style = MaterialTheme.typography.displayLarge)
        Text(if (score == total) "Perfect score!" else "Nice try!")
        Spacer(Modifier.height(24.dp))
        Button(onClick = onRestart) { Text("Play again") }
    }
}

@Preview(showBackground = true)
@Composable
fun QuizScreenPreview() {
    QuizScreen(QuizUiState(selectedOption = 1), onOptionSelected = { }, onNextClick = { }, onRestart = { })
}

@Preview(showBackground = true)
@Composable
fun QuizResultPreview() {
    QuizScreen(QuizUiState(score = 3, isFinished = true), { }, { }, { })
}
```

Notice the two previews: one mid-quiz with option 1 selected, and one showing the results. Neither needs a ViewModel.

4. Try it

- Run the app, answer two questions, then **rotate** the phone. The quiz continues where you were, thanks to the ViewModel.
- Add **Log.d** to QuizViewModel's init: it runs once, not on every rotation.
- Put QuizRoute in a NavHost from Week 7, with a Home screen before it.

5. Project challenges

- Add **isAnswerRevealed** to QuizUiState: after Next, show green/red for 1 second (viewModelScope.launch + delay) before moving on.
- Add a **timer**: 15 seconds per question, counting down in the ViewModel. When it reaches 0, move to the next question.
- Keep a list of wrong answers in QuizUiState and show them on the result screen.
- Write down which functions you would test in QuizViewModel (you'll write the tests in Week 14).

Week 8 checklist

I can...	Done?
Explain MVVM and unidirectional data flow	[ ]
Write a ViewModel and get it with viewModel()	[ ]
Expose state with a private MutableStateFlow and a public StateFlow	[ ]
Collect state with collectAsStateWithLifecycle()	[ ]
Design a UiState data class (and a sealed one when needed)	[ ]
Split screens into Route (stateful) and Screen (stateless)	[ ]
Read navigation arguments in a ViewModel with SavedStateHandle	[ ]
Show loading, errors and one-off messages	[ ]
Rebuild the Quiz app with a ViewModel	[ ]

Quick Quiz

Q1. Where does the quiz score live now?

Q2. Why does the quiz survive rotation without rememberSaveable?

Q3. Why is QuizScreen easy to preview?

Q4. What does return@update state do?

Q5. Why are currentQuestion and progress computed properties?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Why ViewModel? MVVM

Exercise 1

```
class LikesViewModel : ViewModel() {
    var likes by mutableStateOf(0)
        private set
    fun like() { likes++ }
    fun unlike() { if (likes > 0) likes-- }
}

@Composable
fun LikesScreen(viewModel: LikesViewModel = viewModel()) {
    Column(Modifier.padding(16.dp)) {
        Text("${viewModel.likes} likes")
        Row {
            Button(onClick = viewModel::like) { Text("Like") }
            TextButton(onClick = viewModel::unlike) { Text("Unlike") }
        }
    }
}
```

Exercise 2

```
class WaterViewModel : ViewModel() {
    var ml by mutableStateOf(0)
        private set
    fun addGlass() { ml += 250 }
    fun reset() { ml = 0 }
}
```

Exercise 3

```
// A ViewModel lives longer than the Activity (it survives rotation). If it kept a reference
// to the old Activity, that destroyed Activity (and all its views) could never be freed from
// memory: a memory leak.
class GoodViewModel : ViewModel()
```

Quiz answers

Q1. Configuration changes such as rotation.

Q2. Model (data layer), View (composables) and ViewModel.

Q3. So only the ViewModel can change it; the UI can only read it.

Q4. The same ViewModel instance as before.

Q5. The ViewModel outlives the Activity, so it would keep a destroyed Activity in memory (a leak).

Day 2: StateFlow

Exercise 1

```
class Thermostat {
    private val _temperature = MutableStateFlow(22)
    val temperature: StateFlow<Int> = _temperature.asStateFlow()
    fun up() = _temperature.update { (it + 1).coerceAtMost(30) }
    fun down() = _temperature.update { (it - 1).coerceAtLeast(16) }
}

fun main() {
    val t = Thermostat()
    repeat(10) { t.up() }
    println(t.temperature.value)
}
```

Exercise 2

```
class LikesViewModel : ViewModel() {
    private val _likes = MutableStateFlow(0)
    val likes: StateFlow<Int> = _likes.asStateFlow()
    fun like() = _likes.update { it + 1 }
}

@Composable
fun LikesScreen(viewModel: LikesViewModel = viewModel()) {
    val likes by viewModel.likes.collectAsStateWithLifecycle()
    Button(onClick = viewModel::like) { Text("$likes likes") }
}
```

Exercise 3

```
// Starting at 25 (printed), then 25 again (ignored: same value), then 26 (printed):
// 2 lines. StateFlow only emits when the value actually changes.
fun answer() = 2
```

Quiz answers

Q1. MutableStateFlow can be changed; StateFlow is read-only.

Q2. So the UI can only observe the state, never change it.

Q3. Sets a new value calculated from the current one, safely.

Q4. No, only real changes are emitted.

Q5. val x by flow.collectAsStateWithLifecycle()

Day 3: Designing UiState

Exercise 1

```
data class ProfileUiState(
    val name: String = "",
    val bio: String = "",
    val isSaving: Boolean = false,
    val errorMessage: String? = null
) {
    val canSave: Boolean get() = name.isNotBlank() && !isSaving
}
```

Exercise 2

```
data class ProfileUiState(val name: String = "", val bio: String = "", val isSaving: Boolean = false)

class ProfileViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(ProfileUiState())
    val uiState: StateFlow<ProfileUiState> = _uiState.asStateFlow()

    fun onNameChange(name: String) = _uiState.update { it.copy(name = name) }
    fun onBioChange(bio: String) = _uiState.update { if (bio.length <= 150) it.copy(bio = bio) else it }
    fun onSaveClick() = _uiState.update { it.copy(isSaving = true) }
}
```

Exercise 3

```
data class ProfileUiState(val name: String = "", val bio: String = "", val isSaving: Boolean = false) {
    val canSave: Boolean get() = name.isNotBlank() && !isSaving
}

@Composable
fun ProfileScreen(uiState: ProfileUiState, onNameChange: (String) -> Unit,
                  onBioChange: (String) -> Unit, onSaveClick: () -> Unit) {
    Column(Modifier.padding(16.dp)) {
        OutlinedTextField(uiState.name, onNameChange, label = { Text("Name") })
        OutlinedTextField(uiState.bio, onBioChange, label = { Text("Bio") })
        Button(onClick = onSaveClick, enabled = uiState.canSave) {
            Text(if (uiState.isSaving) "Saving..." else "Save")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun ProfileEmptyPreview() { ProfileScreen(ProfileUiState(), { }, { }, { }) }

@Preview(showBackground = true)
@Composable
fun ProfileSavingPreview() { ProfileScreen(ProfileUiState("Anjana", "Android dev", isSaving = true), { }, { }, { }) }
```

Quiz answers

Q1. One source of truth, consistent updates, and the screen collects only one thing.

Q2. The state is immutable; changes create a new copy with copy().

Q3. Derived from other fields (a computed property), not stored separately.

Q4. When the screen shows completely different content in each state, one at a time.

Q5. It only takes uiState and lambdas, so you can create any state directly.

Day 4: Route + Screen and the ViewModel Lifecycle

Exercise 1

```
class StepsViewModel : ViewModel() {
    private val _steps = MutableStateFlow(0)
    val steps: StateFlow<Int> = _steps.asStateFlow()
    fun add(n: Int) = _steps.update { it + n }
}

@Composable
fun StepsRoute(viewModel: StepsViewModel = viewModel()) {
    val steps by viewModel.steps.collectAsStateWithLifecycle()
    StepsScreen(steps = steps, onAdd = viewModel::add)
}

@Composable
fun StepsScreen(steps: Int, onAdd: (Int) -> Unit) {
    Column(Modifier.padding(16.dp)) {
        Text("Steps: $steps")
        Button(onClick = { onAdd(10) }) { Text("+10") }
    }
}

@Preview(showBackground = true)
@Composable
fun StepsScreenPreview() { StepsScreen(steps = 120, onAdd = { }) }
```

Exercise 2

```
@Serializable data class ProductDetailRoute(val productId: Int)

class ProductDetailViewModel(savedStateHandle: SavedStateHandle) : ViewModel() {
    private val productId = savedStateHandle.toRoute<ProductDetailRoute>().productId
    private val _title = MutableStateFlow("Product $productId")
    val title: StateFlow<String> = _title.asStateFlow()
}
```

Exercise 3

```
class LoggingViewModel : ViewModel() {
    init { println("init") }            // use Log.d("VM", "init") on Android
    override fun onCleared() { println("onCleared") }
}
// Rotating: nothing is logged (same ViewModel). Pressing Back: onCleared appears.
```

Quiz answers

Q1. Get the ViewModel, collect its state and pass state and events to the stateless screen.

Q2. From SavedStateHandle: savedStateHandle.toRoute().

Q3. No. It's cleared only when the screen leaves the back stack for good.

Q4. No. Use a database or DataStore for data that must survive closing the app.

Q5. onCleared(), and viewModelScope is cancelled.

Day 5: Loading, Errors & One-Off Events

Exercise 1

```
class AuthRepository {
    suspend fun login(email: String, password: String): Boolean {
        delay(1000)
        return password != "wrongpass"
    }
}

data class LoginUiState(val email: String = "", val password: String = "",
                        val isLoading: Boolean = false, val errorMessage: String? = null)

class LoginViewModel(private val repository: AuthRepository = AuthRepository()) : ViewModel() {
    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    fun onLoginClick() {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            val ok = repository.login(_uiState.value.email, _uiState.value.password)
            _uiState.update {
                it.copy(isLoading = false, errorMessage = if (ok) null else "Incorrect email or password")
            }
        }
    }
}
```

Exercise 2

```
@Composable
fun WeatherError(message: String, onRetry: () -> Unit) {
    Column {
        Text(message, color = MaterialTheme.colorScheme.error)
        Button(onClick = onRetry) { Text("Retry") }
    }
}
```

Exercise 3

```
data class LoginUiState(val isLoggedIn: Boolean = false, val userMessage: String? = null)

class LoginViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    fun onLoginSuccess() = _uiState.update { it.copy(isLoggedIn = true, userMessage = "Welcome back!") }
    fun onMessageShown() = _uiState.update { it.copy(userMessage = null) }
}
```

Quiz answers

Q1. It runs in the background without freezing the UI, and is cancelled when the ViewModel is cleared.

Q2. Set isLoading = false.

Q3. Put the message in UiState, show it in LaunchedEffect, then call onMessageShown() to clear it.

Q4. The ViewModel updates state; the screen reacts and calls a lambda; the NavHost navigates.

Q5. Runs a coroutine when the composable appears and again whenever the key changes.

Day 6: Weekly Project: Quiz App with a ViewModel

Quiz answers

Q1. In QuizUiState, inside QuizViewModel.

Q2. The ViewModel survives configuration changes and keeps the state.

Q3. It's stateless: it only takes a QuizUiState and lambdas.

Q4. Returns the unchanged state from the update lambda when no option is selected.

Q5. They're derived from other state, so they can never be out of sync.

> **Next:** When you finish this week, tell your mentor and ask for the **Week 9** material. Great work, Anjana!